

Foundations of Cryptography, Fall 2025
Problem Set 4
Due Monday, November 24

Total Number of Points: 100

Collaboration Policy: Collaboration is permitted and encouraged in small groups of at most three students. You are free to collaborate in discussing answers, but you must write up solutions on your own, and must specify in your submission the names of any collaborators. Do not copy any text from your collaborators; the writeup must be entirely your work. Do not write down solutions on a board and copy it verbatim; again, the writeup must be entirely your own words and your own work and should demonstrate clear understanding of the solution. **Solutions should be typeset in L^AT_EX.** You may make use of material referenced on the course website, provided that you clearly acknowledge all sources/tools used. Of course, scavenging for solutions from prior years is forbidden.

On the Use of LLMs: You may use AI however you wish to deepen your understanding of the lecture material. Upload the notes, talk to your AI about them, ask for more explanation or examples; it's all fine. **You may not use AI/LLMs in any way to work on your homework.** You may not upload assignments, ask for hints, ask how certain concepts from the lectures might be applied to specific homework problems, or upload your assignments to check for correctness or clarity or anything else. You may not include any AI-generated content whatsoever in your homework submissions. If it becomes clear that you have used an AI tool when working on your homework (either directly by making edits or to ask for hints/solutions), we may mark your grade down to reflect that.

1 Perfectly Binding Hash Functions: An Oxymoron? (20 points)

A hash function $H : \{0, 1\}^n \rightarrow \{0, 1\}^m$ compresses its n -bit input into an m -bit output for some $m < n$. A perfectly binding function $H : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is injective, i.e. given $H(x)$, x is uniquely determined. Could one have a perfectly binding hash function? Clearly not! Nevertheless, in this problem, we will construct hash functions that achieve a variant of perfect binding.

Definition 1. A bit-wise perfectly binding hash function is a pair of polynomial-time algorithms (Gen, H) where

- $hk \leftarrow \text{Gen}(1^\lambda, i)$ takes as input an integer $i \in [n(\lambda)]$ and outputs a public hashing key hk ;
- $y = H(hk, x)$ is a deterministic algorithm that takes an $n(\lambda)$ -bit input x and produces an $m(\lambda)$ -bit output, where $m(\lambda) < n(\lambda)$ for all sufficiently large λ .

We additionally require the following two properties:

1. **Index Hiding.** For every pair of indices $i_0, i_1 \in [n(\lambda)]$, the hash keys $hk_0 \leftarrow \text{Gen}(1^\lambda, i_0)$ and $hk_1 \leftarrow \text{Gen}(1^\lambda, i_1)$ are computationally indistinguishable.
2. **Bit-wise Perfectly Binding.** For any $i \in [n(\lambda)]$, $hk \leftarrow \text{Gen}(1^\lambda, i)$, and $x, x' \in \{0, 1\}^{n(\lambda)}$, if $H(hk, x) = H(hk, x')$, then $x[i] = x'[i]$.

1. (10 points) Recall the Quadratic Residuosity Assumption from the third problem set:

Assumption 1 (Quadratic Residuosity Assumption for Blum Integers). For any poly-size algorithm A , its advantage in solving the Quadratic Residuosity Problem is negligible in the security parameter λ . That is, for random $p \neq q$ of λ bits each where $p, q \equiv 3 \pmod{4}$, and random $x \in \mathbb{Z}_N^*$ with $(\frac{x}{N}) = 1$,

$$\Pr[A(N, x) = 1 \text{ if } x \text{ is a residue, and } 0 \text{ otherwise}] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Construct a bit-wise perfectly binding hash function, assuming the hardness of the quadratic residuosity problem. Prove that your construction is correct.

2. (10 points) Construct a bit-wise perfect binding hash function, assuming the existence of a fully homomorphic encryption (FHE) scheme $(\text{Gen}_{\text{FHE}}, \text{Enc}_{\text{FHE}}, \text{Dec}_{\text{FHE}}, \text{Eval}_{\text{FHE}})$ with perfect correctness. Prove that your construction is correct.

You may also assume that $(\text{Gen}_{\text{FHE}}, \text{Enc}_{\text{FHE}}, \text{Dec}_{\text{FHE}}, \text{Eval}_{\text{FHE}})$ is *truly compact*. More precisely, this means there exists some polynomial $p(\lambda)$ such that the following statement holds: for all polynomial-size circuits C which output a single bit, all inputs $\mu_1, \dots, \mu_n \in \{0, 1\}$, and all keys $(pk, sk) \leftarrow \text{Gen}_{\text{FHE}}(1^\lambda)$, $|\text{Eval}_{\text{FHE}}(pk, C, \{\text{Enc}(pk, \mu_i)\}_{i=1}^n)| \leq p(\lambda)$.

2 Statistically Hiding Commitments (15 points)

In lecture, we have seen a construction of computationally hiding and statistically binding commitments, which can be used to construct computational zero-knowledge proofs. In this problem, we consider how to construct statistically hiding and computationally binding commitments, which can be used to construct statistical zero-knowledge arguments.

We will work with the discrete logarithm assumption over a family of groups $\{G_\lambda\}_{\lambda \in \mathbb{N}}$, where each G_λ is a group of prime order q_λ :

Assumption 2 (DLOG for $\{G_\lambda\}_{\lambda \in \mathbb{N}}$). *Let g_λ be a uniformly random generator of G_λ .¹ For every poly-size adversary $\mathcal{A}$, there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$,*

$$\Pr_{x \leftarrow \mathbb{Z}_{q_\lambda}^*, g} [A(1^\lambda, g, g^x) \equiv x \bmod q] \leq \mu(\lambda),$$

where $q = q_\lambda$ and $g = g_\lambda$.

DLOG-Based Commitment Scheme

- **Gen**(1^λ): Sample uniformly random generators $g = g_\lambda$ and $h = h_\lambda$ from G_λ . Output $\text{pp} = (g, h)$.
- **Com**(pp, m): Parse $\text{pp} = (g, h)$. Sample random $r \leftarrow \mathbb{Z}_q$ and output $\text{com} = g^m \cdot h^r$. The opening is r .
- **Ver**($\text{pp}, \text{com}, m, r$): Parse $\text{pp} = (g, h)$. Output 1 iff $g^m \cdot h^r = \text{com}$.

It is not hard to see that the commitment scheme has perfect correctness: that is, if the committer and receiver behave as prescribed, the receiver will always accept the committer's opening. We now take a look at two other properties of this commitment scheme.

1. (5 points) Show that the commitment scheme is *perfectly hiding*: for all $\lambda \in \mathbb{N}$ and any two messages $m_0, m_1 \in \mathbb{Z}_{q_\lambda}$,

$$(\text{pp}, \text{Com}(\text{pp}, m_0)) \equiv (\text{pp}, \text{Com}(\text{pp}, m_1)),$$

where $\text{pp} \leftarrow \text{Gen}(1^\lambda)$.

2. (10 points) Assuming the discrete logarithm is hard in $\{G_\lambda\}_{\lambda \in \mathbb{N}}$, show that the commitment scheme is *computationally binding*: for all poly-size adversaries $\mathcal{A}$, there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$,

$$\Pr_{\text{pp} \leftarrow \text{Gen}(1^\lambda)} \left[\begin{array}{l} m_0 \not\equiv m_1 \bmod q_\lambda \wedge \\ \text{Ver}(\text{pp}, \text{com}, m_0, r_0) = 1 \wedge \\ \text{Ver}(\text{pp}, \text{com}, m_1, r_1) = 1 \end{array} : (\text{com}, m_0, r_0, m_1, r_1) \leftarrow \mathcal{A}(1^\lambda, \text{pp}) \right] \leq \mu(\lambda).$$

¹Note that since G_λ is of prime order (and hence cyclic), a uniformly random generator is simply a uniformly random (non-identity) element.

3 Does Zero Knowledge Compose in Parallel? (15 points)

We have seen in class that assuming one-way functions exist, there is a three message zero knowledge proof system for an NP-complete problem (graph 3-colorability) with soundness $\frac{1}{\text{poly}(n)}$, and that the soundness can be amplified by repeating this proof system many times in sequence to obtain a zero knowledge proof system with negligible soundness. Unfortunately, this results in a very long protocol. Wouldn't it be nice if we could instead repeat the three message protocol many times *in parallel*? Then maybe we could get negligible soundness with only three messages!

Unfortunately, the problem is that the resulting proof system (which is actually sound!) may not be zero knowledge! In this problem, we will see how repeating an interactive proof can ruin zero knowledge. Consider the following zero knowledge proof Π for the discrete logarithm:

- Public input: a string $h = g^x$ (along with a group $\mathbb{G}$ and generator g); prover input: the discrete logarithm x .
- First message: P chooses a uniformly random r and sends $R = g^r$ to V .
- Second message: V chooses a uniformly random bit b .
- Third message: P sends $z = r + bx$ to V . V accepts if $g^z = R \cdot h^b$.

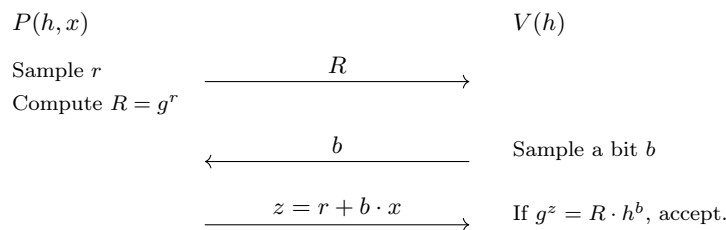


Figure 2: The Zero Knowledge Proof Π for Discrete Log.

The Counterexample We will now consider the following strange modification $\tilde{\Pi}$ of Π (see Figure 3):

- Public input: a string $h = g^x$ (along with a group $\mathbb{G}$ and generator g); prover input: the discrete logarithm x .
- First message: V makes a uniformly random guess x^* and checks if $g^{x^*} = h$. If this happens, V sets a variable **mode** to 1, and otherwise V sets **mode** to 0. V then sends **mode** to P .
- If **mode** = 0, P and V proceed to execute Π λ times in sequence. If **mode** = 1, V instead proves to P (λ times) that V knows x , which is equal to x^* ! That is, V and P execute Π (λ times) with their roles reversed. If V convinces P that V knows x , P then sends x to V in the clear.

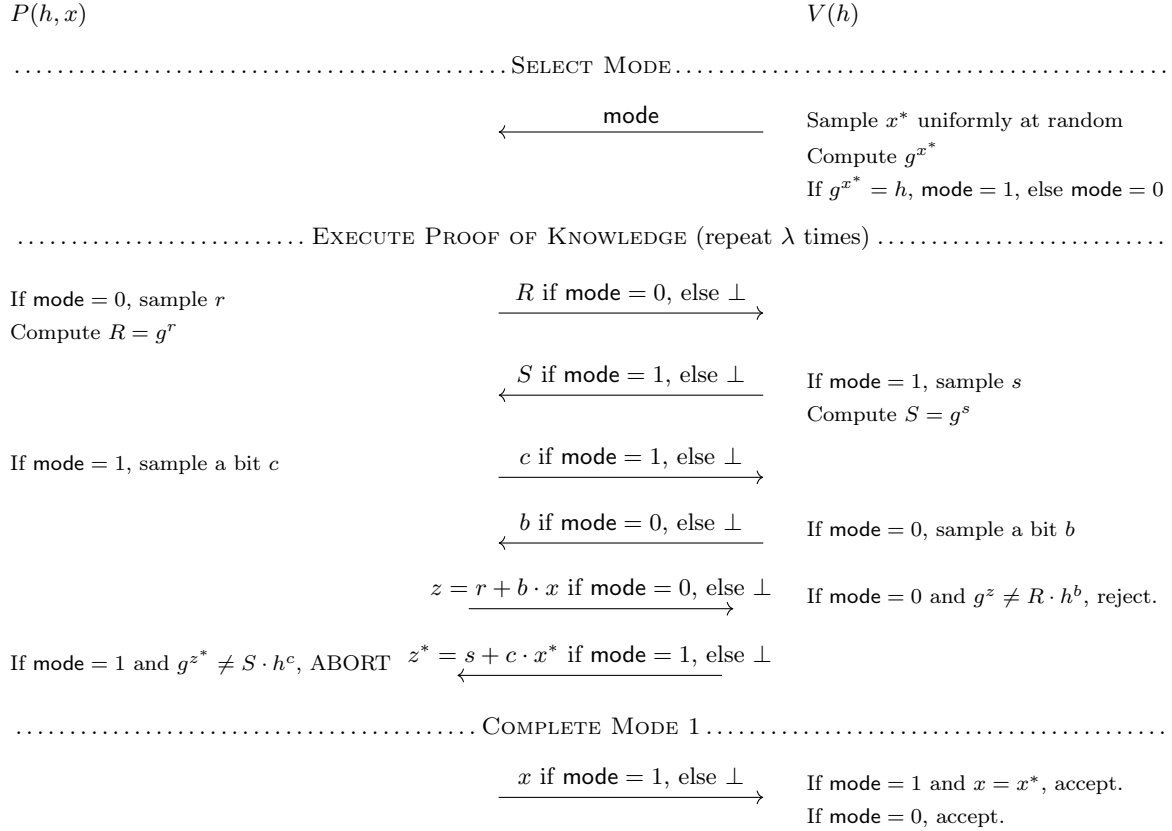


Figure 3: The Protocol $\tilde{\Pi}$.

One can show that $\tilde{\Pi}$ is still a zero knowledge proof for discrete logarithm (we will not ask you to do this). **Instead, prove that if the discrete logarithm problem is hard, then $\tilde{\Pi}$ executed two times in parallel is *not* zero knowledge! In particular, show how a cheating verifier can learn the discrete logarithm x with certainty.**

4 But Really, Does Zero Knowledge Compose in Parallel? (25 points)

We saw in the previous problem that zero knowledge is not always preserved when a protocol is executed many times in parallel. However, the example seems contrived, and we would like to say something about parallel composition anyway! To do so, we will introduce a new (weaker) notion of security against malicious verifiers, which we call witness indistinguishability (WI).

Definition 2. A proof system Π for a language $L \in \text{NP}$ is witness indistinguishable if for every (efficient) malicious verifier V^* , every instance $x \in L$, every pair of witnesses w_1, w_2 for x , and every auxiliary input z ,

$$\text{view}_{V^*} \langle P(w_1), V^*(x, z) \rangle \approx_c \text{view}_{V^*} \langle P(w_2), V^*(x, z) \rangle.$$

In other words, V^ cannot tell which witness an honest prover P is using in an execution of the protocol, even given arbitrary auxiliary information about (x, w_1, w_2) (possibly including both w_1 and w_2).*

As a motivating example, consider the following scenario. Suppose that there is some organization whose membership is a known list of public keys $\text{pk}_1, \dots, \text{pk}_n$. Each member i of the organization has secret key sk_i . Now, suppose that a member of the organization wants to prove to an outside party (say, a journalist) that they belong to the organization *without revealing* which member they are. Witness indistinguishable proofs allow you to do this: you are trying to prove a statement with n possible witnesses (the secret keys), and you do not want to reveal which witness you have.

1. (10 points) **Zero Knowledge Proofs are Witness Indistinguishable.** Prove that every (auxiliary input) zero knowledge proof system Π (for a language $L \in \text{NP}$) is also witness indistinguishable.
2. (15 points) **Composition of WI Proofs.** From part 1, we see that the three-message protocol for 3-colorability from lecture is witness indistinguishable. However, it still has poor soundness. Prove that (three-message) WI proof systems actually compose under parallel composition!² That is, if Π is a three-message WI proof system for some language L with soundness μ and completeness 1, prove that Π^t (a t -wise parallel repetition of Π) is a WI proof system for L with soundness μ^t and completeness 1.

²In fact, composition holds for WI proof systems with an arbitrary polynomial number of rounds, but we will restrict to three-message proofs for simplicity.

5 The Power of Homomorphism (25 points)

In this problem, we will see that homomorphism is quite a powerful tool—often, homomorphic properties already present enough structure to imply public-key encryption!

5.1 PKE from re-randomizable encryption (10 points)

Recall the definition of re-randomizable encryption, which is a property that most homomorphic encryption schemes tend to have:

Definition 3 (Re-randomizable secret-key encryption). *A randomized secret-key encryption scheme $\text{SKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ over message space $\mathcal{M} = \{0, 1\}$ is re-randomizable if there exists a PPT algorithm ReRand such that for every key $k \leftarrow \text{Gen}(1^\lambda)$, message $m \in \mathcal{M}$, and encryption $c \leftarrow \text{Enc}(k, m)$,*

$$\text{ReRand}(c) \stackrel{c}{\approx} \text{Enc}(k, m),^3$$

and $\text{Dec}(k, \text{ReRand}(c)) = \text{Dec}(k, c)$ for all ciphertexts c .

Suppose there exists a re-randomizable secret key-encryption scheme $(\text{Gen}, \text{Enc}, \text{Dec}, \text{ReRand})$. Show that there exists a (CPA-secure) public-key encryption scheme $(\text{Gen}', \text{Enc}', \text{Dec}')$.

5.2 PKE from additive homomorphism (15 points)

Let $\text{SKE} = (\text{Gen}, \text{Enc}, \text{Dec}, \text{Add})$ be a *secret-key* encryption scheme for single-bit messages that satisfies the standard compactness and security guarantees, and supports homomorphic addition:

- **Additive homomorphism mod 2.** Add is a deterministic algorithm such that for any ciphertexts $\{c_i\}_{i \in [n]}$,

$$\text{Dec}(k, \text{Add}(c_1, \dots, c_n)) = \bigoplus_{i=1}^n \text{Dec}(k, c_i).$$

Now, define a public-key encryption scheme $\text{PKE} = (\text{Gen}', \text{Enc}', \text{Dec}')$ as follows:

- $\text{Gen}'(1^\lambda)$: Let $\ell = \ell(\lambda)$ be a polynomial to be specified later. Sample $k \leftarrow \text{Gen}(1^\lambda)$. For each $i \in [\ell]$, sample fresh

$$X_i \leftarrow \text{Enc}(k, 0), \quad Y_i \leftarrow \text{Enc}(k, 1).$$

Output $\text{pk} = ((X_1, \dots, X_\ell), (Y_1, \dots, Y_\ell))$ and $\text{sk} = k$.

- $\text{Enc}'(\text{pk}, m)$: Parse $\text{pk} = ((X_1, \dots, X_\ell), (Y_1, \dots, Y_\ell))$. Sample a uniformly random subset $S \subseteq [\ell]$ conditioned on $|S| \equiv m \pmod{2}$. Form the ℓ -tuple Z by setting $Z_i := Y_i$ if $i \in S$ and $Z_i := X_i$ if $i \notin S$. Output

$$c := \text{Add}(Z_1, \dots, Z_\ell).$$

- $\text{Dec}'(\text{sk}, c)$: Parse $\text{sk} = k$ and output $\text{Dec}(k, c)$.

Suppose the ciphertexts $c \leftarrow \text{Enc}(k, m), k \leftarrow \text{Gen}(1^\lambda)$ satisfy $|c| \leq n(\lambda)$ for some polynomial n . Specify the polynomial ℓ such that PKE is CPA-secure, and prove correctness and security. You may take the following information-theoretic statement for granted.

Theorem 1. *Let $X_1, \dots, X_\ell$ and $Y_1, \dots, Y_\ell$ be i.i.d. over a finite set. Sample a uniformly random bit $\sigma \in \{0, 1\}$ and let $S \subseteq [\ell]$ be uniformly random subject to $|S| \pmod{2} \equiv \sigma$. Define $Z_i := X_i$ for $i \notin S$ and $Z_i := Y_i$ for $i \in S$. Let $X := (X_1, \dots, X_\ell)$, $Y := (Y_1, \dots, Y_\ell)$, and $Z := (Z_1, \dots, Z_\ell)$. For any (possibly unbounded) adversary that sees X, Y , and any n -bit (possibly randomized) function $g(Z)$ of Z , its advantage in predicting $\sigma \equiv |S| \pmod{2}$ is at most $2^{-\ell/5+n+1}$.⁴*

³The ciphertext on the right-hand side is a *fresh* encryption of m with the key k which is not necessarily equal to c .

⁴For this theorem to make sense, $g(Z)$ cannot depend on X, Y , and Z together (or else it can simply leak σ). Thus, the function g is restricted to depend only on the bits in Z (and not X, Y , and Z simultaneously). Of course, g can depend on the *distribution* of X and Y .